

TFM: SLM with Transformers and GRU

Guillem Gonzalo Silveira

April 2026

c. 2846 words

Abstract

This thesis explores the engineering lifecycle of developing a Small Language Model (SLM) from scratch, utilizing a hybrid architecture of Transformers and Gated Recurrent Units (GRU). In a landscape dominated by massive models, the primary barrier for custom development is the extreme economic cost. This research shifts the focus from model scale to the complete deployment pipeline. We document the implementation of CUDA-optimized code, orchestration on AWS EC2 environments, and data strategies for conversational agents. Furthermore, we integrate Retrieval-Augmented Generation (RAG) to provide the model with external, up-to-date information, significantly reducing hallucinations without the prohibitive cost of full-scale retraining. Through performance and cost-benefit analysis using *spot instances*, we evaluate the feasibility of local development versus cloud scaling. Finally, we demonstrate production readiness by deploying the system on Kubernetes, visualized and managed through ArgoCD.

Abstract

Este trabajo de fin de máster investiga el ciclo de vida de ingeniería en el desarrollo de un Small Language Model (SLM) desde cero, integrando arquitecturas Transformer y GRU. En un entorno donde el desarrollo de modelos de lenguaje está limitado por altos costes operativos, este proyecto se centra en el flujo completo de ingeniería más allá de la escala del modelo. La investigación abarca el desarrollo de código optimizado para CUDA, la gestión de infraestructura en AWS EC2 y la preparación de datos para agentes conversacionales. Se incluye además la implementación de tecnologías RAG (Retrieval-Augmented Generation), que permiten al modelo consultar fuentes externas de información para responder con precisión sin necesidad de reentrenamientos costosos. Mediante el uso de *spot instances* y subconjuntos de datos, se analizan los costes y el rendimiento del sistema. Finalmente, se valida la puesta en producción mediante un despliegue en Kubernetes gestionado con ArgoCD.

Abstract

Aquest treball de fi de màster investiga el cicle de vida d'enginyeria en el desenvolupament d'un Small Language Model (SLM) des de zero, integrant arquitectures Transformer i GRU. En un entorn on el desenvolupament de models de llenguatge està limitat per alts costos operatius, aquest projecte se centra en el flux complet d'enginyeria més enllà de l'escala del model. La investigació inclou el desenvolupament de codi optimitzat per a CUDA, la gestió d'infraestructura a AWS EC2 i la preparació de dades per a agents conversacionals. S'hi inclou a més la implementació de tecnologies RAG (Retrieval-Augmented Generation), que permeten al model consultar fonts externes d'informació per respondre amb precisió sense la necessitat de reentrenaments costosos. Mitjançant l'ús de *spot instances* i subconjunts de dades, s'analitzen els costos i el rendiment del sistema. Finalment, es valida la posada en producció mitjançant un desplegament a Kubernetes gestionat amb ArgoCD.

Contents

1	Introducción	6
1.1	Motivación	6
1.2	Objetivos	6
2	Estado del Arte	7
2.1	Evolución de los Modelos de Lenguaje: De LLM a SLM	7
2.2	Limitaciones de los Transformers Puros	7
2.3	Arquitecturas Híbridas y Recurrencia Moderna	7
2.4	Aumento por Recuperación (RAG)	7
3	Fundamentos Teóricos	9
3.1	Transformers y el Mecanismo de Atención	9
3.2	Gated Recurrent Units (GRU)	9
3.3	Retrieval-Augmented Generation (RAG)	9
4	Diseño e Implementación de la Arquitectura Híbrida	10
4.1	Estrategia de Curación y Mezcla de Datos (Data Mixing)	10
4.1.1	Selección de Datasets de Propósito General	10
4.1.2	Infraestructura de Datos: Formato Apache Arrow	10
4.1.3	Inyección de Conocimiento de Dominio (Fase Final)	11
4.1.4	Distribución del Dataset de Pre-entrenamiento	11
4.2	Arquitectura del Modelo	11
4.3	Optimización con CUDA	12
5	Infraestructura Cloud y Análisis de Costes	13
5.1	Ecosistema AWS: ECS y ECR	13
5.2	Estrategia de Spot Instances	13
5.2.1	Identificación y Etiquetado	13
5.2.2	Selección de la Imagen de Máquina (AMI)	13
5.2.3	Selección del Hardware: Tipo de Instancia	15
5.2.4	Configuración Técnica y Automatización	17
5.3	Evaluación de Rendimiento vs Coste	17

6	MLOps: Despliegue y Puesta en Producción	18
6.1	Pruebas Automatizadas y Calidad de Código	18
6.2	Orquestación con Kubernetes	18
6.3	GitOps con ArgoCD	18
6.4	Integración del Sistema RAG	18
7	Conclusiones y Trabajo Futuro	19
7.1	Viabilidad de los SLM Híbridos	19
7.2	Lecciones Aprendidas	19
7.3	Trabajo Futuro	19

1 Introducción

En la actualidad, los modelos de lenguaje de gran escala (LLM) han revolucionado la inteligencia artificial. Sin embargo, su desarrollo y entrenamiento están limitados a unas pocas organizaciones debido a los prohibitivos costes económicos y de computación. Este proyecto nace de la necesidad de explorar alternativas viables: los Small Language Models (SLM).

1.1 Motivación

La democratización de la IA requiere que las organizaciones puedan desarrollar, desplegar y controlar sus propios modelos sin depender de APIs de terceros o inversiones multimillonarias. La capacidad de comparar una solución desarrollada a medida frente a estándares de la industria permite validar la eficiencia de arquitecturas personalizadas.

1.2 Objetivos

El objetivo principal es diseñar e implementar una arquitectura híbrida entre Transformers y GRU, realizando una comparativa directa con la familia de modelos **SmolLM** de Hugging Face. El estudio se centra en:

- **Desarrollo desde cero:** Implementación de una arquitectura híbrida optimizada mediante kernels CUDA.
- **Estrategias de Inyección de Conocimiento:** Evaluación comparativa de dos vertientes: *Retrieval-Augmented Generation* (RAG) frente a *Fine-tuning* mediante *Low-Rank Adaptation* (LoRA).
- **Validación de Dominio:** Uso de un dataset sintético específico de Zurich como fase final para dotar de conocimiento especializado a ambos modelos.
- **Pipeline de Ingeniería:** Gestión del ciclo de vida completo, desde la infraestructura en AWS (Spot Instances) hasta el despliegue en Kubernetes mediante GitOps y ArgoCD.

2 Estado del Arte

Este capítulo analiza el panorama actual de los modelos de lenguaje, la evolución hacia modelos más eficientes y las investigaciones previas en arquitecturas híbridas.

2.1 Evolución de los Modelos de Lenguaje: De LLM a SLM

En los últimos años, el campo del Procesamiento del Lenguaje Natural (NLP) ha estado dominado por los *Large Language Models* (LLM), cuya arquitectura base sigue siendo el Transformer propuesto por Vaswani et al. [1]. Modelos como Llama 2 [2] han demostrado capacidades emergentes sorprendentes al escalar el número de parámetros a miles de millones.

Sin embargo, recientemente ha surgido una tendencia hacia los *Small Language Models* (SLM), impulsada por la necesidad de eficiencia y despliegue en entornos con recursos limitados. Investigaciones como "*Textbooks Are All You Need*" [3] demuestran que la calidad de los datos puede compensar el tamaño del modelo, permitiendo que arquitecturas de menos de 3 mil millones de parámetros (como Phi-3 o Mistral) compitan en tareas específicas con modelos mucho más grandes. En este contexto, la familia de modelos SmolLM [4] representa el estado actual de la técnica en SLMs ultra-eficientes, optimizando tanto el proceso de entrenamiento como la curación de datos para maximizar el rendimiento en dispositivos de consumo.

2.2 Limitaciones de los Transformers Puros

A pesar de su éxito, los Transformers convencionales presentan un cuello de botella fundamental: la complejidad cuadrática del mecanismo de auto-atención respecto a la longitud de la secuencia [1]. Esto implica un consumo de memoria de video (VRAM) y un tiempo de cómputo que crece de forma prohibitiva para contextos largos. En un entorno de producción, esto se traduce en altos costes operativos y latencias elevadas, lo que motiva la búsqueda de arquitecturas con escalado lineal.

2.3 Arquitecturas Híbridas y Recurrencia Moderna

Para mitigar las limitaciones del Transformer, han resurgido las arquitecturas basadas en recurrencia, pero con enfoques modernos. Las *Gated Recurrent Units* (GRU) [5] ofrecen un manejo eficiente de secuencias con un estado oculto de tamaño fijo, eliminando el coste cuadrático.

Investigaciones recientes como Mamba [6] han propuesto modelos de espacio de estados (*State Space Models*) que logran un rendimiento comparable a los Transformers pero con inferencia de tiempo lineal. La tendencia actual, y el núcleo de este TFM, es la hibridación: combinar capas de atención para capturar relaciones globales con capas recurrentes o lineales para mantener la eficiencia secuencial.

2.4 Aumento por Recuperación (RAG)

Finalmente, el estado del arte no solo se centra en la arquitectura del modelo, sino en su interacción con fuentes externas. El *Retrieval-Augmented Generation* (RAG) [7] se

ha consolidado como el estándar para reducir alucinaciones en modelos de lenguaje. Esta técnica permite que el modelo consulte bases de datos externas en tiempo real, proporcionando respuestas basadas en evidencia sin necesidad de reentrenamientos costosos, un componente vital para asistentes técnicos especializados.

3 Fundamentos Teóricos

En este capítulo se analizan las tecnologías base que componen la propuesta híbrida.

3.1 Transformers y el Mecanismo de Atención

El impacto del mecanismo de *self-attention* en el procesamiento del lenguaje y su capacidad para capturar relaciones globales.

3.2 Gated Recurrent Units (GRU)

La eficiencia de las redes recurrentes en el manejo de secuencias y cómo las puertas de actualización permiten mantener una memoria selectiva con menor coste computacional que las LSTM.

3.3 Retrieval-Augmented Generation (RAG)

El concepto de aumentar la capacidad de respuesta del modelo mediante la recuperación de contexto externo, mitigando alucinaciones y proporcionando datos actualizados.

4 Diseño e Implementación de la Arquitectura Híbrida

Este capítulo detalla la aportación técnica central de este TFM: la integración de Transformers y GRU en un único modelo funcional, así como la estrategia de datos y fine-tuning que permite su entrenamiento eficiente y la comparativa con el estado del arte.

4.1 Estrategia de Curación y Mezcla de Datos (Data Mixing)

La calidad del entrenamiento de un Small Language Model (SLM) depende críticamente de la diversidad y representatividad de los datos. Se ha diseñado una estrategia de *Data Mixing* para la fase de pre-entrenamiento y alineación general, reservando el conocimiento específico para una fase final.

4.1.1 Selección de Datasets de Propósito General

Se han seleccionado cinco pilares de datos abiertos para dotar al modelo de capacidades lingüísticas, de razonamiento y técnicas:

- **OpenAssistant (OASST1):**¹ Fuente principal de naturalidad, proporcionando turnos de diálogo con matices humanos reales.
- **ShareGPT:**² Utilizado para transferir el estilo resolutivo y la estructura de respuesta de modelos avanzados.
- **Alpaca:**³ Aporta una base sólida de *instruction-tuning* para tareas directas.
- **UltraChat:**⁴ Fundamental para garantizar la consistencia en diálogos largos, poniendo a prueba la memoria de la arquitectura híbrida.
- **The Stack (YAML):**⁵ Subconjunto del dataset *The Stack* filtrado para archivos de configuración YAML, esencial para el conocimiento de infraestructura y sintaxis DevOps.

4.1.2 Infraestructura de Datos: Formato Apache Arrow

Para garantizar un acceso eficiente a los datasets durante el entrenamiento, se ha optado por el uso de archivos en formato **Apache Arrow** (`.arrow`). Este estándar, que subyace a la librería *datasets* de Hugging Face utilizada en el proyecto, ofrece ventajas tecnológicas clave para la viabilidad de los SLMs:

- **Carga mediante Memory Mapping:** Los archivos `.arrow` permiten el acceso de tipo “zero-copy”, mapeando los datos directamente del disco a la memoria RAM. Esto permite entrenar con volúmenes de datos que superan la memoria física disponible, algo crítico en el uso de instancias con recursos limitados.

¹<https://huggingface.co/datasets/OpenAssistant/oasst1>

²https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

³<https://huggingface.co/datasets/tatsu-lab/alpaca>

⁴<https://huggingface.co/datasets/stingning/ultrachat>

⁵<https://huggingface.co/datasets/bigcode/the-stack>

- **Almacenamiento Columnar:** A diferencia de formatos tradicionales como JSON o CSV, Arrow organiza la información por columnas. Esto optimiza el procesamiento de secuencias de texto al permitir lecturas parciales y una serialización extremadamente rápida.
- **Consistencia Estructural:** El uso de este formato garantiza que los campos *instruction*, *input* y *output* mantengan una tipificación estricta, facilitando la integración directa con los modelos de datos de Pydantic definidos en la aplicación.

4.1.3 Inyección de Conocimiento de Dominio (Fase Final)

Una vez que el modelo ha adquirido capacidades lingüísticas generales, se procede a dotarlo de conocimiento especializado mediante el dataset sintético de Zurich. Esta fase se aborda desde dos metodologías para su posterior comparativa:

- **Fine-tuning con LoRA (Low-Rank Adaptation):** Entrenamiento eficiente de una pequeña fracción de los parámetros del modelo para integrar el conocimiento directamente en los pesos de la red.
- **Retrieval-Augmented Generation (RAG):** Implementación de una arquitectura de consulta externa que proporciona contexto en tiempo real sin modificar los pesos del modelo.

4.1.4 Distribución del Dataset de Pre-entrenamiento

Para optimizar el rendimiento bajo restricciones de recursos, se ha definido la siguiente distribución de pesos para la fase inicial:

Categoría	Dataset	Peso	Propósito
Conversacional	OpenAssistant / Ultra-Chat	40%	Tono humano y consistencia en diálogos largos
Instrucciones	Alpaca / ShareGPT	40%	Seguimiento de instrucciones y estructura resolutive
Infraestructura	The Stack (YAML)	20%	Sintaxis DevOps y automatización

Table 1: Estrategia de mezcla de datos para el entrenamiento base del SLM.

4.2 Arquitectura del Modelo

Descripción de la sinergia entre las capas de atención y las capas recurrentes. Se detalla cómo la GRU refina la salida de los bloques de atención para mejorar la eficiencia secuencial y reducir la carga computacional en el manejo de estados de memoria.

4.3 Optimización con CUDA

Desarrollo de kernels personalizados para maximizar el uso de la GPU. Se aborda el resto de paralelizar la parte de atención mientras se mantiene la eficiencia en la naturaleza secuencial de la GRU. El diseño de estos kernels se ha optimizado para manejar las longitudes de secuencia y el *padding* definidos en nuestra estrategia de datos, minimizando la latencia en el paso de estados entre la atención global y la recurrencia local.

5 Infraestructura Cloud y Análisis de Costes

Uno de los pilares de este trabajo es la demostración de la viabilidad económica del proyecto mediante el uso de infraestructura cloud optimizada.

5.1 Ecosistema AWS: ECS y ECR

Para el despliegue y orquestación del modelo, se ha seleccionado Amazon Web Services (AWS), aprovechando los siguientes servicios:

- **Amazon Elastic Container Registry (ECR):** Actúa como el repositorio privado de imágenes de Docker, integrado totalmente con el flujo de CI/CD de GitHub Actions.
- **Amazon Elastic Container Service (ECS):** Se utiliza para la orquestación de los contenedores. A diferencia de un despliegue en instancias EC2 puras, ECS facilita la gestión del ciclo de vida de la aplicación, el escalado y el autorreparado de tareas.

5.2 Estrategia de Spot Instances

El coste es el principal inhibidor en el desarrollo de modelos de lenguaje. Para mitigar esto, el clúster de ECS se ha configurado para utilizar **Amazon EC2 Spot Instances**. Estas instancias permiten acceder a la capacidad computacional sobrante de AWS con descuentos de hasta el 90% sobre el precio de las instancias bajo demanda.

5.2.1 Identificación y Etiquetado

Siguiendo las buenas prácticas de gestión de infraestructura, cada instancia se identifica mediante etiquetas (*tags*). Se ha definido el nombre **nexus-slm** (asignado a la clave **Name**) como estándar para permitir un seguimiento sencillo de los recursos activos en la consola de AWS y mediante scripts de automatización.

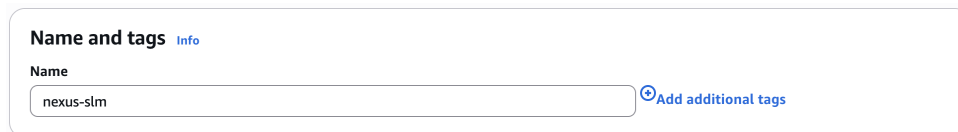


Figure 1: Configuración del nombre de la instancia en la consola de AWS.

5.2.2 Selección de la Imagen de Máquina (AMI)

Para que el software sea capaz de comunicarse eficientemente con el hardware de aceleración, es crítico seleccionar una *Amazon Machine Image* (AMI) que proporcione el entorno base adecuado.

Se ha seleccionado la AMI: **Deep Learning Base AMI with Single CUDA (Amazon Linux 2023)**. Esta imagen viene preconfigurada con los *drivers* de NVIDIA y el *toolkit* de CUDA necesarios para el entrenamiento del modelo, eliminando la complejidad

técnica de la instalación manual y garantizando la compatibilidad entre el kernel de Linux y los núcleos Tensor de la GPU.

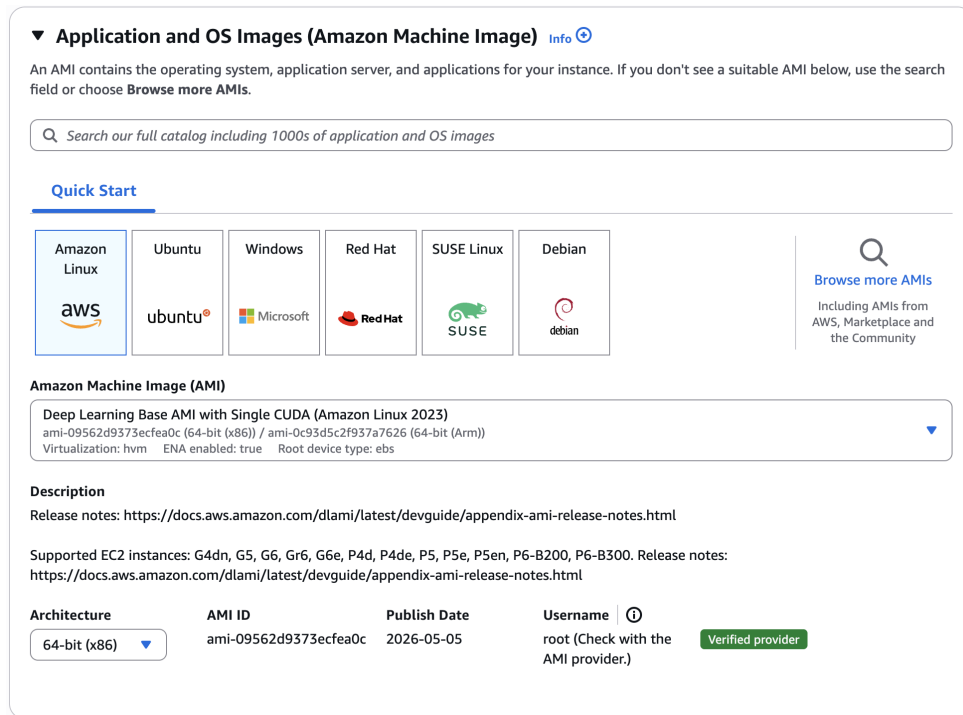


Figure 2: Interfaz de selección de la AMI especializada para Deep Learning con soporte CUDA.

Esta elección habilita el uso de una amplia gama de instancias optimizadas para computación acelerada. A continuación, se detallan las familias de instancias compatibles y sus especificaciones de hardware:

Instancia	Hardware GPU	Propósito Principal
G4dn	NVIDIA T4 (16 GB VRAM)	Inferencia y entrenamiento ligero
G5	NVIDIA A10G (24 GB VRAM)	Entrenamiento de modelos SLM / Gráficos
G6 / Gr6	NVIDIA L4 (24 GB VRAM)	Eficiencia energética y IA generativa
G6e	NVIDIA L40S (48 GB VRAM)	LLMs de tamaño medio y LLM Fine-tuning
P4d / P4de	NVIDIA A100 (40/80 GB VRAM)	Cómputo de alto rendimiento (HPC)
P5 / P5e / P5en	NVIDIA H100 (80 GB VRAM)	Entrenamiento de modelos de lenguaje masivos
P6-B200 / B300	NVIDIA Blackwell	Próxima generación de IA de ultra-escala

Table 2: Familias de instancias EC2 soportadas por la AMI de Deep Learning y su hardware asociado.

5.2.3 Selección del Hardware: Tipo de Instancia

Tras definir el entorno de software, el siguiente paso crítico es la elección del hardware subyacente. El tipo de instancia de EC2 determina la capacidad de CPU, memoria ram, almacenamiento y rendimiento de red del host que ejecutará el modelo.

Para este proyecto, se ha seleccionado la familia de instancias **G6**. Esta elección se fundamenta en la necesidad de disponer de una gran capacidad de memoria de video (VRAM) y un ancho de banda de memoria elevado, factores indispensables para almacenar y procesar los pesos de la arquitectura Transformer y los estados ocultos de la GRU durante el entrenamiento.

Consultando las especificaciones técnicas oficiales de AWS, observamos que la familia G6 ofrece diversas configuraciones escalables basadas en procesadores AMD EPYC 7R13 y aceleradores NVIDIA L4:

G6							
g6.xlarge	16.00	AMD EPYC 7R13	4	2	2	1 x NVIDIA L4 GPU	22 GiB (1 x 22 GiB)
Instance type	Memory (GiB)	Processor	vCPUs	CPU cores	Threads per core	Accelerators	Accelerator memory
g6.2xlarge	32.00	AMD EPYC 7R13	8	4	2	1 x NVIDIA L4 GPU	22 GiB (1 x 22 GiB)
g6.4xlarge	64.00	AMD EPYC 7R13	16	8	2	1 x NVIDIA L4 GPU	22 GiB (1 x 22 GiB)
g6.8xlarge	128.00	AMD EPYC 7R13	32	16	2	1 x NVIDIA L4 GPU	22 GiB (1 x 22 GiB)
g6.12xlarge	192.00	AMD EPYC 7R13	48	24	2	4 x NVIDIA L4 GPU	89 GiB (4 x 22 GiB)
g6.16xlarge	256.00	AMD EPYC 7R13	64	32	2	1 x NVIDIA L4 GPU	22 GiB (1 x 22 GiB)
g6.24xlarge	384.00	AMD EPYC 7R13	96	48	2	4 x NVIDIA L4 GPU	89 GiB (4 x 22 GiB)
g6.48xlarge	768.00	AMD EPYC 7R13	192	96	2	8 x NVIDIA L4 GPU	178 GiB (8 x 22 GiB)

Figure 3: Especificaciones de hardware para las diferentes variantes de la familia de instancias G6.

La versatilidad de la serie G6 permite desde configuraciones económicas con una sola GPU (como la `g6.xlarge` con 22 GB de VRAM) hasta potentes nodos de cómputo con múltiples aceleradores, proporcionando el equilibrio ideal entre coste y rendimiento para

un Small Language Model.

5.2.4 Configuración Técnica y Automatización

Para garantizar la estabilidad del entrenamiento bajo el modelo de subasta de Spot, se han implementado las siguientes configuraciones técnicas adicionales:

- **Launch Templates:** Definición de plantillas de lanzamiento que especifican las familias detalladas anteriormente, permitiendo que el clúster elija dinámicamente según disponibilidad.
- **Capacity Providers:** Gestión del escalado automático de las instancias de infraestructura.
- **Gestión de Interrupciones:** Señales de terminación de 2 minutos para realizar el guardado automático (*checkpointing*) en Amazon S3.

Esta estrategia es ideal para las fases de entrenamiento y fine-tuning del SLM, donde la tolerancia a interrupciones es gestionada mediante la capacidad de ECS para relanzar tareas automáticamente en nuevas instancias disponibles.

5.3 Evaluación de Rendimiento vs Coste

Presentación de gráficas y métricas que comparan el tiempo de ejecución con el gasto económico real, demostrando la eficiencia de la arquitectura híbrida en entornos de recursos limitados.

6 MLOps: Despliegue y Puesta en Producción

El ciclo de vida del modelo culmina con su despliegue en un entorno de producción real.

6.1 Pruebas Automatizadas y Calidad de Código

La fiabilidad del pipeline de datos y de la arquitectura del modelo se garantiza mediante una suite de pruebas unitarias implementada con el framework `pytest`.

Para evitar la dependencia de servicios externos y minimizar los tiempos de ejecución, se utiliza la técnica de *mocking* (objetos simulados). Esto permite validar la lógica de adquisición de datos, la integridad de los modelos de datos definidos con Pydantic y el flujo principal de la aplicación sin necesidad de realizar conexiones de red o descargas reales. Además, estas pruebas se han integrado en el flujo de desarrollo mediante *hooks* de *pre-commit*, garantizando que ningún cambio de código sea registrado si no supera satisfactoriamente todos los tests, manteniendo una cobertura de código superior al 95%.

6.2 Orquestación con Kubernetes

Configuración del clúster de K8s para servir el modelo híbrido de forma escalable.

6.3 GitOps con ArgoCD

Implementación del flujo de despliegue continuo, permitiendo que cualquier cambio en la configuración se refleje automáticamente en el clúster.

6.4 Integración del Sistema RAG

Despliegue de la infraestructura de base de datos vectorial y su conexión con el modelo para la inferencia aumentada.

7 Conclusiones y Trabajo Futuro

Este capítulo resume los hallazgos del proyecto y las posibles líneas de investigación.

7.1 Viabilidad de los SLM Híbridos

Conclusiones sobre si un modelo de estas características es suficiente para casos de uso empresariales específicos.

7.2 Lecciones Aprendidas

Reflexión sobre los desafíos de ingeniería encontrados durante el desarrollo CUDA y el despliegue MLOps.

7.3 Trabajo Futuro

Posibilidades de mejora: cuantización del modelo, soporte para arquitecturas de inferencia como vLLM u Ollama, y fine-tuning con datos de dominio específico.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [3] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, *et al.*, “Textbooks are all you need,” *arXiv preprint arXiv:2306.11644*, 2023.
- [4] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf, “Smollm - blazingly fast and remarkably powerful.” <https://huggingface.co/blog/smollm>, 2024. Accessed: 2026-05-03.
- [5] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
- [6] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.